

LINQ (Language Integrated Query) Documentation

Vehicle Service Management System

Repository → EF Core DbContext → Database

↓

Service Layer (LINQ Operations)

↓

DTOs returned to Controllers

LINQ Methods Used in This Project

Method	Purpose	Example
Where()	Filter data by condition	<code>.Where(u => u.Role == "Customer")</code>
Select()	Transform/project data	<code>.Select(u => new UserDto { ... })</code>
FirstOrDefault()	Get first match or null	<code>.FirstOrDefault(u => u.UserId == id)</code>
GroupBy()	Group data by key	<code>.GroupBy(sr => sr.Status)</code>
Sum()	Calculate total	<code>.Sum(p => p.AmountPaid)</code>
Count()	Count items	<code>.Count(sr => sr.Status == "Completed")</code>
Average()	Calculate average	<code>.Average(b => b.TotalAmount)</code>
OrderBy() / OrderByDescending()	Sort data	<code>.OrderByDescending(sr => sr.RequestedDate)</code>
Take() / Skip()	Pagination	<code>.Skip(10).Take(10)</code>
Any()	Check if any match exists	<code>.Any(b => b.ServiceRequestId == id)</code>
ToList()	Execute and materialize	<code>.ToList()</code>

Method	Purpose	Example
ToDictionary()	Convert to dictionary	.ToDictionary(c => c.CategoryId)
ToHashSet()	Convert to HashSet	.ToHashSet()
Distinct()	Remove duplicates	.Distinct()
Contains()	Check if item exists	.Contains(id)

Real LINQ Code Snippets from This Project

1. FILTERING - Dynamic Where Clauses

File: Services/ServiceRequestService.cs

// Dynamic filtering based on filter criteria

```
var query = serviceRequests.AsEnumerable();
```

// Filter by Status

```
if (!string.IsNullOrEmpty(filter.Status))
```

```
    query = query.Where(r => r.Status == filter.Status);
```

// Filter by Priority

```
if (!string.IsNullOrEmpty(filter.Priority))
```

```
    query = query.Where(r => r.Priority == filter.Priority);
```

// Filter by Customer ID

```
if (filter.CustomerId.HasValue)
```

```
    query = query.Where(r => r.CustomerId == filter.CustomerId.Value);
```

// Filter by Category ID

```
if (filter.CategoryId.HasValue)
```

```
    query = query.Where(r => r.CategoryId == filter.CategoryId.Value);
```

```

// Filter by Vehicle ID
if (filter.VehicleId.HasValue)
    query = query.Where(r => r.VehicleId == filter.VehicleId.Value);

// Filter by Date Range
if (filter.FromDate.HasValue)
    query = query.Where(r => r.RequestedDate >= filter.FromDate.Value);

if (filter.ToDate.HasValue)
    query = query.Where(r => r.RequestedDate <= filter.ToDate.Value);

```

2. GROUPBY - Services Grouped by Status

File: Services/ReportService.cs

```

// Group service requests by status and calculate percentage
var totalServices = serviceRequests.Count();
var servicesByStatus = serviceRequests
    .GroupBy(sr => sr.Status)
    .Select(g => new ServiceByStatusDto
    {
        Status = g.Key.ToString(),
        Count = g.Count(),
        Percentage = totalServices > 0
            ? Math.Round(((decimal)g.Count() / totalServices * 100, 2)
            : 0
    })
    .ToList();

```

3. GROUPBY - Monthly Revenue Report

File: Services/ReportService.cs

```
// Group payments by Year and Month, calculate revenue

var monthlyRevenues = payments

    .Where(p => p.PaymentStatus == PaymentStatus.Paid && p.PaymentDate.Year ==
targetYear)

    .GroupBy(p => new { p.PaymentDate.Year, p.PaymentDate.Month })

    .Select(g =>
{
    var serviceCount = bills

        .Where(b => payments.Any(p => p.BillId == b.BillId &&
            p.PaymentDate.Year == g.Key.Year &&
            p.PaymentDate.Month == g.Key.Month))

        .Select(b => b.ServiceRequestId)

        .Distinct()

        .Count();

    return new MonthlyRevenueDto
    {
        Year = g.Key.Year,
        Month = g.Key.Month,
        MonthName =
CultureInfo.CurrentCulture.DateTimeFormat.GetMonthName(g.Key.Month),
        Revenue = g.Sum(p => p.AmountPaid),
        ServiceCount = serviceCount
    };
})

.OrderBy(m => m.Month)

.ToList();
```

4. GROUPBY - Revenue by Category

File: Services/ReportService.cs

// Group service requests by category and calculate revenue

var revenueByCategory = serviceRequests

.Where(sr => paidBills.Any(b => b.ServiceRequestId == sr.ServiceRequestId))

.GroupBy(sr => sr.CategoryId)

.Select(g =>

{

var category = categories.FirstOrDefault(c => c.CategoryId == g.Key);

var categoryBills = paidBills.Where(b =>

g.Any(sr => sr.ServiceRequestId == b.ServiceRequestId));

var categoryRevenue = categoryBills.Sum(b => b.TotalAmount);

return new RevenueByCategoryDto

{

CategoryId = g.Key,

CategoryName = category?.CategoryName ?? "Unknown",

Revenue = categoryRevenue,

ServiceCount = g.Count(),

Percentage = totalRevenueAmount > 0

? Math.Round(categoryRevenue / totalRevenueAmount * 100, 2)

: 0

};

})

.ToList();

5. AGGREGATION - Revenue Calculations

File: Services/DashboardService.cs

// Revenue calculations using Where and Sum

```
var paidPayments = payments.Where(p => p.PaymentStatus == PaymentStatus.Paid);
```

```
var totalRevenue = paidPayments.Sum(p => p.AmountPaid);
```

```
var todayRevenue = paidPayments
```

```
    .Where(p => p.PaymentDate.Date == today)
```

```
    .Sum(p => p.AmountPaid);
```

```
var weekRevenue = paidPayments
```

```
    .Where(p => p.PaymentDate.Date >= startOfWeek)
```

```
    .Sum(p => p.AmountPaid);
```

```
var monthRevenue = paidPayments
```

```
    .Where(p => p.PaymentDate.Date >= startOfMonth)
```

```
    .Sum(p => p.AmountPaid);
```

```
var pendingPayments = bills.Sum(b => b.TotalAmount) - payments.Sum(p =>
p.AmountPaid);
```

6. AGGREGATION - Dashboard Statistics

File: Services/ReportService.cs

// Service status summary using Count with conditions

```
var statusSummary = new ServiceStatusSummaryDto
```

```
{
```

```
    Pending = serviceRequests.Count(sr =>
```

```

        sr.Status == ServiceStatus.Requested ||
        sr.Status == ServiceStatus.Assigned),
    InProgress = serviceRequests.Count(sr =>
        sr.Status == ServiceStatus.InProgress),
    Completed = serviceRequests.Count(sr =>
        sr.Status == ServiceStatus.Completed ||
        sr.Status == ServiceStatus.Closed),
    Cancelled = serviceRequests.Count(sr =>
        sr.Status == ServiceStatus.Cancelled),
    TotalActive = serviceRequests.Count(sr =>
        sr.Status != ServiceStatus.Completed &&
        sr.Status != ServiceStatus.Closed &&
        sr.Status != ServiceStatus.Cancelled)
};

```

7. COUNT - Entity Statistics

File: Services/ReportService.cs

// Count entities with conditions

```

TotalCustomers = users.Count(u => u.Role == UserRole.Customer),
TotalVehicles = vehicles.Count(),
TotalTechnicians = users.Count(u => u.Role == UserRole.Technician && u.IsActive),
LowStockPartsCount = parts.Count(p => p.QuantityInStock <= p.ReorderLevel)

```

```

    return new ServiceRequestDto { /* ... mapping ... */ };
};

Year = vehicle.Year,

OwnerName = customer != null
    ? $"{customer.FirstName}{customer.LastName}"

```

: "Unknown"

8. SUM - Total Calculations

File: Services/ServiceRequestService.cs

// Calculate total estimated cost from selected categories

totalEstimatedCost = selectedCategories.Sum(c => c.BasePrice);

File: Services/ServicePartService.cs

// Calculate total parts charge

return serviceParts.Sum(sp => sp.TotalPrice);

File: Services/PaymentService.cs

// Calculate total paid amount

var totalPaid = existingPayments.Sum(p => p.AmountPaid);

var remainingBalance = bill.TotalAmount - totalPaid;

9. ORDERBY - Sorting Results

File: Services/ServiceAssignmentService.cs

// Sort assignments by date descending

var filteredAssignments = query

.OrderByDescending(a => a.AssignedDate)

.ToList();

File: Services/ReportService.cs

// Sort today's scheduled services by time

.OrderBy(s => s.ScheduledDate)

.ToList();

.

10. DISTINCT - Remove Duplicates

File: Services/ReportService.cs


```
// Get distinct paid bill IDs
var paidBillIds = totalPaidPayments.Select(p => p.BillId).Distinct();

// Count pending bills
PendingCount = bills.Count() -
    payments.Select(p => p.BillId).Distinct().Count()
```

11. SERVICE ASSIGNMENT FILTERING

File: Services/ServiceAssignmentService.cs

```
// Dynamic filtering for service assignments
if (filter.TechnicianId.HasValue)
    query = query.Where(a => a.TechnicianId == filter.TechnicianId.Value);

if (!string.IsNullOrEmpty(filter.Status))
    query = query.Where(a => a.Status == filter.Status);

if (filter.ServiceRequestId.HasValue)
    query = query.Where(a => a.ServiceRequestId == filter.ServiceRequestId.Value);

if (filter.FromDate.HasValue)
    query = query.Where(a => a.AssignedDate >= filter.FromDate.Value);

if (filter.ToDate.HasValue)
    query = query.Where(a => a.AssignedDate <= filter.ToDate.Value);

var filteredAssignments = query
    .OrderByDescending(a => a.AssignedDate)
```

```
.ToList();
```

12. USER SERVICE - Technician Statistics

File: Services/UserService.cs

```
// Get technicians with workload info
```

```
return technicians.Select(t => new TechnicianDto
```

```
{
```

```
    UserId = t.UserId,
```

```
    FullName = $"{t.FirstName} {t.LastName}",
```

```
    Email = t.Email,
```

```
    PhoneNumber = t.PhoneNumber,
```

```
    ActiveAssignments = assignments.Count(a =>
```

```
        a.TechnicianId == t.UserId &&
```

```
        (a.Status == "Assigned" || a.Status == "InProgress")),
```

```
    CompletedAssignments = assignments.Count(a =>
```

```
        a.TechnicianId == t.UserId &&
```

```
        a.Status == "Completed")
```

```
});
```

13. NOTIFICATION SERVICE - User Filtering

File: Services/InAppNotificationService.cs

```
// Get user IDs by role
```

```
var targetUserIds = users
```

```
    .Where(u => u.Role == targetRole && u.IsActive && u.IsEmailVerified)
```

```
    .Select(u => u.UserId)
```

```
    .ToList();
```